MONEU Security Analysis: Stolen Wallet Without the Noise File

18 October 2026 · natusor · natusor@tutamail.com

Summary

An attacker who holds a stolen `wallet.dat` and its correct password but not the noise file cannot move the coins, even with a modified node or a custom noise generator. Every route to a valid spend needs either the original noise file or a preimage of SHA-256, which is out of reach for classical and quantum computers alike.

The practical risk in this scenario is not cryptographic: it is a leftover copy of the original encrypted noise file that was never wiped from the victim's disk, because the attacker already knows the password that decrypts it. The analysis was done on the MONEU source code at commit 867bcee.

Scenario and attacker assumptions

The attacker had physical access to the victim's computer and is given every advantage except the noise file.

Attacker has	Attacker does not have
A copy of `wallet.dat` with all private keys	The noise file `noise.dat`
The correct wallet password, seen while the victim typed it	Any leaf of the noise file that was never spent
Full knowledge of the MONEU code and the public blockchain	Control over what honest nodes accept
The ability to modify and run their own node and generator	

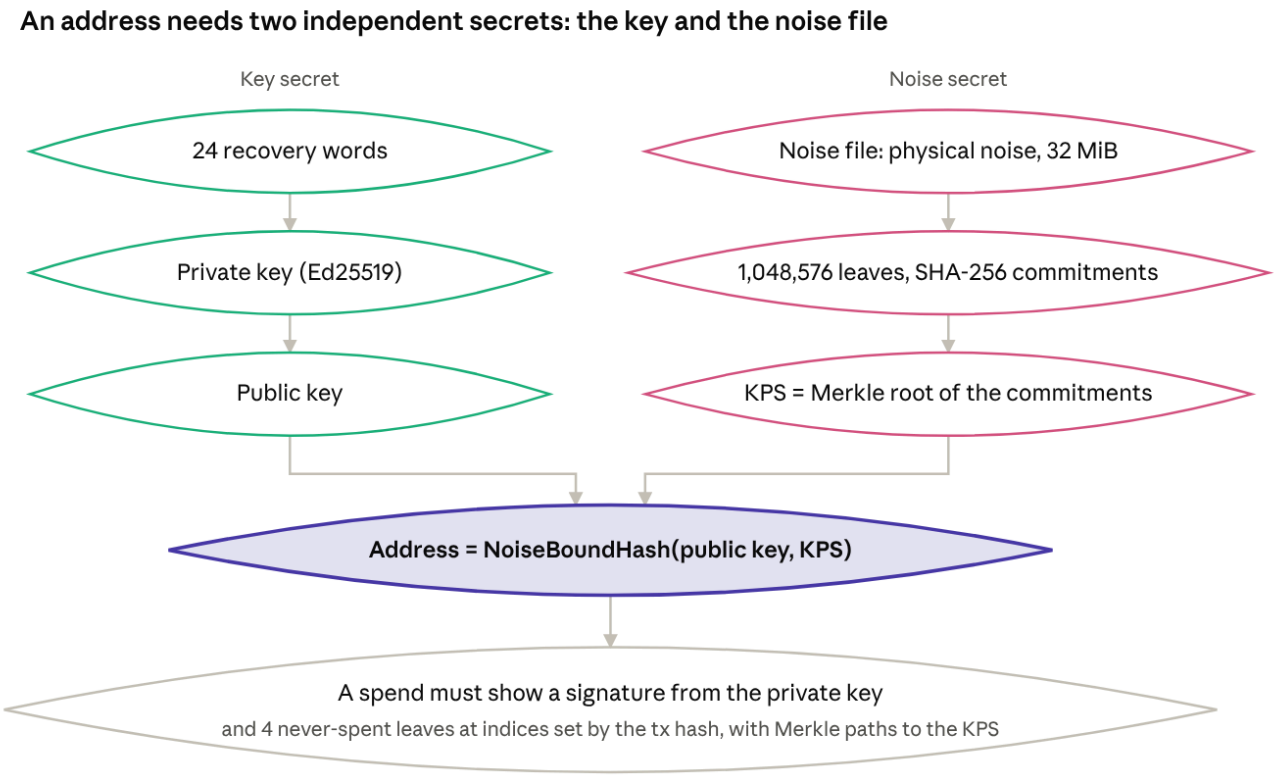
We assume the attacker learned the password by physically watching the victim type it. The victim kept the noise file on a separate storage device, so it was never in the wallet folder the attacker copied. The attacker does not change consensus rules, address prefixes or network magic bytes.

We assume the attacker builds a so-called soft fork of the MONEU node with a modified noise file generator. The generator is given everything the attacker can learn: the stolen private keys, the public keys, the addresses, the KPS published on the chain, the leaves already spent and their Merkle paths. Its task is to produce a noise file that fits the stolen keys, so the attacker can sign and prove a spend from the victim's addresses.

How an address is bound to the noise file

A MONEU address is the hash of two independent secrets, so the key alone can never spend from it. The address check is in `transaction.cpp`, and the noise proofs are checked in

`tx_validation.cpp` and `noise_otp.cpp`.



How a MONEU address binds the key and the noise file

The two columns meet only in the address hash; nothing in the key column says anything about the noise column.

What the noise file is made of

The noise file is 32 MiB of unpredictable data measured from real physical events on the machine that creates it: the tiny timing differences of the processor and memory under load, the hardware temperature sensor, and the operating system's own entropy pool. The generator mixes these measurements together with SHA-512, and it refuses to run on a machine where the processor's timing does not vary, which is why it needs a real computer and takes a few minutes.

SHA-256 appears only afterwards, to build the Merkle tree that turns the file into the single public value KPS. It is not what produces the noise. This matters for the quantum question: even a computer that could break SHA-256 could not recreate the file, because the physical measurements it was built from are stored nowhere and are not a function of anything public. They happened once, on one machine, and were never recorded outside the file itself.

Attack 1: generating a new noise file from the keys

This attack fails because the noise file is not derived from the keys, so the keys carry no information about it. The private keys come from the 24 recovery words, while the noise file is built from unpredictable physical events on the machine that created it, never from the keys. The only link between them is the address hash, `NoiseBoundHash(public key, KPS)`.

A modified generator that reads the keys, addresses and KPS can only output new random data. New data builds a new Merkle tree with a different root KPS. A different KPS gives a different address, which holds no coins, and the address check in `transaction.cpp` rejects it for the victim's coins.

For a generated file to match the victim's address, its tree must end in exactly the same KPS. There are only two ways to get there:

1. **Recreate the original leaves.** The blockchain holds only hashes, so this means reversing SHA-256, which is finding a preimage. That is 2^256 attempts, or 2^128 for a quantum computer. Not feasible.
2. **Invent different leaves that give the same KPS.** This is a second preimage of SHA-256, which is just as infeasible.

Attack 2: forging leaves of the original noise file

This attack fails because the chain stores only hashes of leaves, and turning a hash back into its leaf means breaking SHA-256. Once the victim has spent once, the KPS is public in that input, so the attacker knows the root of the right tree.

Every spend still needs 4 leaves at indices derived from the hash of that same transaction (`DeriveLeafIndices` in `tx_validation.cpp`). For each one the node checks three things in `noise_otp.cpp`:

1. The leaf is bound to this transaction hash.
2. `SHA256(leaf)` is the commitment at that index.
3. The Merkle path from that commitment ends in the KPS.

The attacker cannot choose the indices, and the unspent leaves are not on the chain at all. Producing one valid leaf is a preimage of SHA-256, about 2^256 operations classically and 2^128 with a quantum computer, and a single input needs four.

Attack 3: reusing leaves already revealed on chain

This is the cleverest route, and consensus closes it by rejecting any leaf that was ever spent before. Every past spend by the victim reveals 4 real leaves with their Merkle paths, and those stay public on the chain.

The attacker could keep changing their own transaction, for example its fee, until its hash happens to derive 4 indices that are all already revealed. With a few hundred revealed leaves out of 1,048,576, that search would take only hours on a GPU.

It does not help, because every node checks `IsNoiseLeafSpent(KPS, index)` against the spent-leaf set of the whole chain (checked in `tx_validation.cpp`, kept in `utxo_set.cpp`). A transaction that uses any leaf already spent is rejected with "uses a leaf that is already spent", and a leaf repeated inside one transaction or block is rejected too.

So the already-revealed leaves are useless twice over: the attacker cannot steer a transaction onto them, and even an exact match is refused because the leaf is already marked spent. Recreating a spent leaf changes nothing, because what blocks it is the spent mark, not the difficulty of obtaining the leaf.

Attack 4: a modified node or so-called soft fork

A modified node only fools the attacker's own computer, because every honest node still checks every transaction itself. The attacker can patch their node to accept a transaction without valid noise proofs, while keeping the same prefixes and magic bytes so it still connects to the network.

Accepting what the rules forbid makes the rules looser, and a looser rule is a hard fork, not a soft fork. A real soft fork can only tighten rules, which never lets an invalid spend through. Every honest node rejects the attacker's transaction and any block that contains it, no matter how much hash power the attacker has.

The attacker ends up alone on their own chain, and the victim's coins on the real MONEU chain stay untouched.

The real risk: noise file remnants and malware

The attacker's realistic path is to obtain the real noise file, not to generate one. In this scenario that is easier than it looks, because `noise.dat` is encrypted with the same password as `wallet.dat`, and the attacker already has that password.

- **Deleted copies on disk.** If the noise file was generated on this computer and then only moved or deleted, its sectors often still linger on the disk, and so with physical access an undelete tool may read some of them back, while the known password would decrypt whatever is recovered. This is far from guaranteed, because the noise file is 32 MiB of high-entropy random bytes with no redundancy, and so any sector that was overwritten or not recovered cannot be reconstructed from the rest, which often leaves an incomplete and unusable file. Even so, it is safer to assume the worst and erase the remnants.
- **Malware at spend time.** To send coins, the owner must connect the storage device that holds the noise file. A program already running on the computer can copy it at that moment.

Neither is a weakness of the consensus or the cryptography. Both are about how the file is kept, and the noise file cannot protect against a computer the attacker already controls.

When the generator is weakened on purpose

Everything above holds for the MONEU generator as shipped, which uses real physical entropy and takes no seed. There is one way to lose the protection, and it is entirely in the owner's own hands.

If an owner rewrites the generator so that it builds the noise file from a reproducible seed, for example a passphrase or a fixed number, then the file is no longer unpredictable. Anyone who learns that seed could rebuild the exact same file, and together with stolen keys could then spend. This is not a weakness of MONEU. The stock generator never does this and refuses low-entropy timing sources; only the owner can introduce the flaw by deliberately sabotaging their own tool. The lesson is to always create the noise file with the official generator.

Recommendations

The protocol needs no change for this scenario. The remaining steps are about how users keep the file plus one optional wallet improvement.

For users:

- Keep `noise.dat` and `wallet.dat` on separate devices, and keep a backup of both offline.
- After moving the noise file off the computer, securely erase what is left, for example with `shred` on Linux. On SSDs that is unreliable, so full-disk encryption from the start is the stronger choice.
- Send coins only from a computer you trust, since malware can copy the noise file while its device is connected.
- Use a strong password, because it protects both files.

Possible wallet improvement, outside consensus:

- A separate password for the noise file. Today both files share one password so a stolen password plus a leftover copy of the original `noise.dat` is enough. Two passwords would add an independent barrier.